

The Salesforce Developer's Guide to the Spring '26 Release

Dynamic Event Listeners (**lwc:on**)

Scenario:

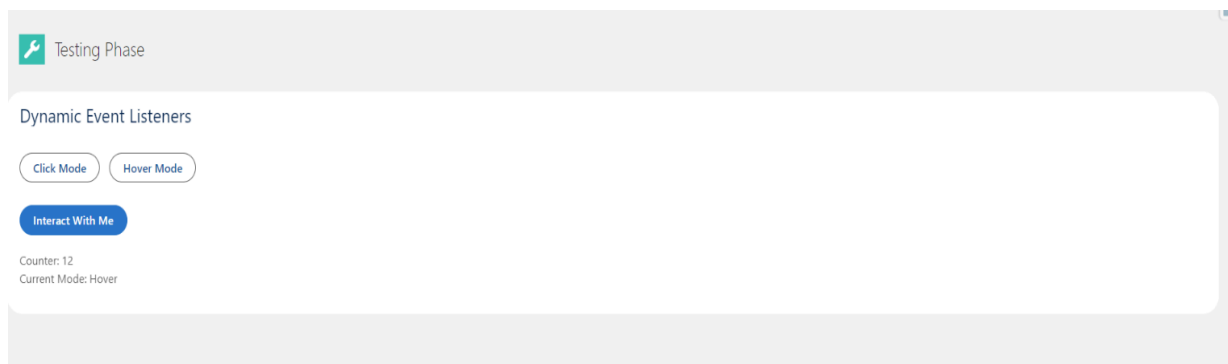
A button behaves differently based on the selected mode:

- Click Mode → Clicking the button increments a counter.
- Hover Mode → Hovering over the button increments the counter.

```
dynamicEventListener.html X JS dynamicEventListener.js X
LightningWebComponent > dynamicEventListener > dynamicEventListener.html
1 <template>
2   <lightning-card title="Dynamic Event Listeners">
3
4     <div class="slds-p-around_medium">
5
6       <lightning-button
7         label="Click Mode"
8         onclick={enableClickMode}>
9     </lightning-button>
10
11     <lightning-button
12       class="slds-m-left_small"
13       label="Hover Mode"
14       onclick={enableHoverMode}>
15 </lightning-button>
16
17     <div class="slds-m-top_large">
18       <button
19         class="slds-button slds-button_brand"
20         lwc:on={buttonHandlers}>
21         Interact With Me
22       </button>
23     </div>
24
25     <p class="slds-m-top_medium">
26       Counter: {counter}
27     </p>
28
29     <p>
30       Current Mode: {mode}
31     </p>
32
33   </div>
34 </lightning-card>
35 </template>
```

```
per tools ▾ Utility tools ▾ Themes ▾
< dynamicEventListener.html × JS dynamicEventListener.js ×
LightningWebComponent > dynamicEventListener > dynamicEventListener.js
1 import { LightningElement } from 'lwc';
2
3 export default class DynamicEventListener extends LightningElement {
4
5     counter = 0;
6     mode = 'Click';
7
8     enableClickMode() {
9         this.mode = 'Click';
10    }
11
12    enableHoverMode() {
13        this.mode = 'Hover';
14    }
15
16    incrementCounter = () => {
17        this.counter++;
18    };
19
20    get buttonHandlers() {
21        if (this.mode === 'Click') {
22            return {
23                click: this.incrementCounter
24            };
25        }
26
27        return {
28            mouseenter: this.incrementCounter
29        };
30    }
31 }
```

Output:



GraphQL mutations in LWC

```
<template>

  <lightning-card title="Create Account Using GraphQL Mutation">

    <div class="slds-p-around_medium">

      <lightning-input
        label="Account Name"
        value={name}
        onchange={handleName}>
      </lightning-input>

      <lightning-input
        label="Phone"
        value={phone}
        onchange={handlePhone}>
      </lightning-input>

      <lightning-input
        label="Industry"
        value={industry}
        onchange={handleIndustry}>
      </lightning-input>

      <br/>

      <lightning-button
        variant="brand"
        label="Create Account"
        onclick={createAccount}
        disabled={loading}>
      </lightning-button>

      <template lwc:if={loading}>
        <lightning-spinner
          alternative-text="Loading">
        </lightning-spinner>
      </template>
    </div>
  </lightning-card>
</template>
```

```

        </lightning-spinner>
    </template>

</div>

</lightning-card>

</template>

import { LightningElement } from 'lwc';

import { gql, executeMutation } from 'lightning/graphql';

import { ShowToastEvent } from 'lightning/platformShowToastEvent';

const CREATE_ACCOUNT = gql`

mutation CreateAccount($input: AccountCreateInput!) {

    uiapi {

        AccountCreate(input: $input) {

            Record {

                Id

                Name {
                    value
                }

                Phone {
                    value
                }

                Industry {
                    value
                }
            }
        }
    }
}
`

```

```

        }

    }

}

};

```

```

export default class GraphqlCreateAccount extends LightningElement {

```

```

    name = '';

```

```

    phone = '';

```

```

    industry = '';

```

```

    loading = false;

```

```

    handleName(event) {
        this.name = event.target.value;
    }

```

```

    handlePhone(event) {
        this.phone = event.target.value;
    }

```

```

    handleIndustry(event) {
        this.industry = event.target.value;
    }

```

```

    async createAccount() {

        if (!this.name) {

            this.showToast(
                'Error',
                'Account Name is required',
                'error'
            );

```

```
    );

    return;
}

this.loading = true;

try {

    const result = await executeMutation({

        query: CREATE_ACCOUNT,

        variables: {

            input: {

                Account: {

                    Name: this.name,

                    Phone: this.phone,

                    Industry: this.industry

                }

            }

        }

    });

    const account =
        result.data.uiapi.AccountCreate.Record;

    this.showToast(

        'Success',
```

```

        `Account Created : ${account.Id}`,

        'success'

    );

    this.resetForm();

}
catch(error) {

    this.showToast(

        'Error',

        error.body?.message || error.message,

        'error'

    );

}
finally {

    this.loading = false;

}

}

resetForm() {

    this.name = '';

    this.phone = '';

    this.industry = '';

}

```

```

showToast(title,message,variant){

    this.dispatchEvent(

        new ShowToastEvent({

            title,

            message,

            variant

        })

    );

}

}

```

Output:

Create Account Using GraphQL Mutation

Account Name

Phone

Industry

Create Account

Testing App

Testing Phase

Testing data | Account

Accounts

Recently Viewed

New Import Assign Label

Search this list...

2 items • Updated a few seconds ago

	Account Name	Account Site	Phone	Account Owner Alias
1	Testing data		87654321	LPraj
2	EdgeMX		(512) 757-6000	LPraj

TypeScript support for base components:

We've completed the TypeScript rollout for Lightning base components. You can now get full support for every base component (like lightning-input or lightning-button) via the `@salesforce/lightning-types` package.

Why use it? This package provides IntelliSense and catches errors in your code before you even try to deploy it

Pro-Tip: If you created your own custom types in the past, we recommend switching to the official package for better consistency

In our example LWC component below, we import type definitions for `LightningInput`, `LightningButton`, and `LightningCombobox` from the `@salesforce/lightning-types` package, then cast `event.target` to these types in our event handlers — enabling full autocomplete and type safety for properties like `value`, `validity`, and `label`. This showcases how the completed TypeScript rollout lets you build LWCs while catching errors at compile time rather than runtime.

```
import { LightningElement } from 'lwc';
import LightningInput from 'lightning/input';
import LightningCombobox from 'lightning/combobox';

export default class MyComponent extends LightningElement {
  inputValue: string = '';

  // Cast event.target to the imported type for full IntelliSense
  handleInputChange(event: Event): void {
    const input = event.target as LightningInput;
    this.inputValue = input.value ?? ''; // Autocomplete for .value,
    .validity, .checkValidity(), etc.
  }

  handleComboboxChange(event: Event): void {
    const combobox = event.target as LightningCombobox;
    console.log(combobox.value); // Type-safe access to combobox
    properties
  }
}
```

Tooling setup:

Step 1: Install dependencies: Add TypeScript and the Lightning type definitions to your project: `npm install --save-dev typescript @salesforce/lightning-types`.

Step 2: Enable TypeScript in VS Code: Add this setting to your `.vscode/settings.json`:

```
{  
  "salesforcedx-vscode-lwc.preview.typeScriptSupport": true  
}
```

This automatically generates a `tsconfig.json` in your LWC directory.

Step 3: Compile before deploying: Run the TypeScript compiler to generate JavaScript files: `npx tsc --project ./force-app/main/default/lwc`.

Step 4: Deploy to Salesforce: Deploy the compiled JavaScript files: `sf project deploy start --source-dir force-app`.

Note: The Agentforce 360 Platform stores only JavaScript; your `.ts` files are compiled locally, and the resulting `.js` files are what get deployed to your org.

Navigate to flows from LWC

```
<template>  
  <lightning-card title="Service Request">  
  
    <div class="slds-p-around_medium">  
  
      <template if:false={showFlow}>  
  
        <lightning-button  
          label="Create Service Request"  
          variant="brand"  
          onclick={launchFlow}>  
        </lightning-button>  
  
      </template>  
    </div>  
  </lightning-card>  
</template>
```

```

        <template if:true={showFlow}>

            <lightning-flow
                flow-api-name="Create_Service_Request"
                flow-input-variables={inputVariables}
                onstatuschange={handleStatusChange}>
            </lightning-flow>

        </template>

    </div>

</lightning-card>
</template>

import { LightningElement, api } from 'lwc';
import { NavigationMixin } from 'lightning/navigation';
export default class LaunchServiceRequestFlow extends
NavigationMixin(LightningElement) {

    @api recordId;
    showFlow = false;
    inputVariables = [];

    launchFlow() {

        this.inputVariables = [
            {
                name: 'recordId',
                type: 'String',
                value: this.recordId
            }
        ];

        this.showFlow = true;
    }

    handleStatusChange(event) {

        if (event.detail.status === 'FINISHED') {

```

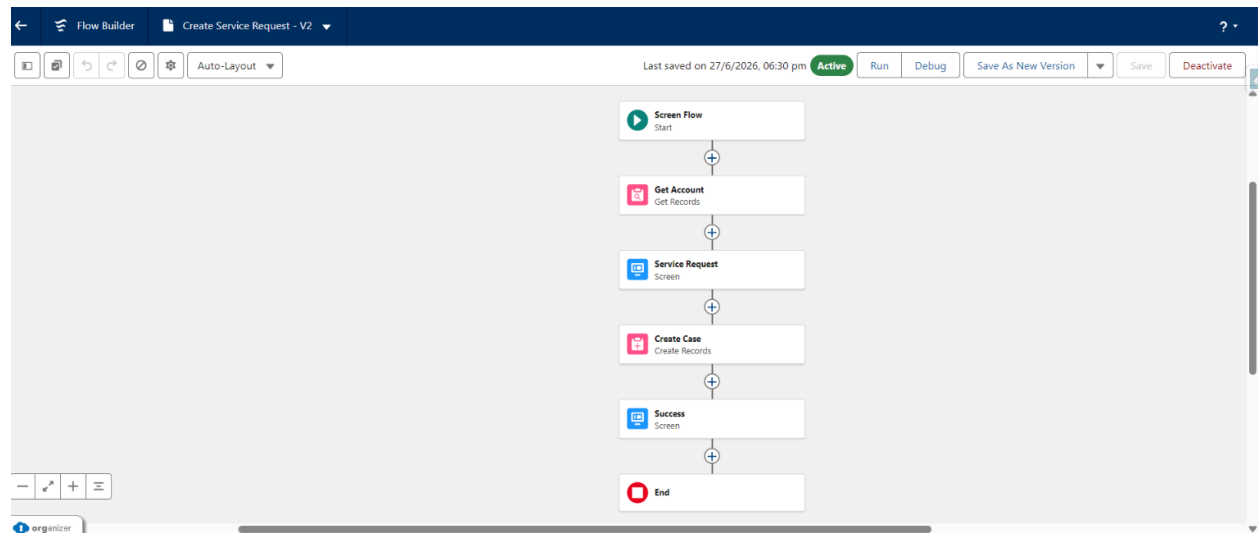
```

        this[NavigationMixin.Navigate]({
            type: 'standard__recordPage',
            attributes: {
                recordId: this.recordId,
                objectApiName: 'Account',
                actionName: 'view'
            }
        });
    }

}

}

```



Output:

Testing App

Testing Phase

Testing data | Account

00001027 | Case

Account

Testing data

+ Follow

Edit

New Contact

New Case

Type

Phone

Website

Account Owner

Account Site

Industry

87654321

Lalji Prajapati

CSE

Related

Details

Assets

Account Owner

Lalji Prajapati

Rating

Account Name

Testing data

Phone

87654321

Parent Account

Fax

Account Number

Website

Account Site

Ticker Symbol

Type

Ownership

Service Request

Create Service Request

Activity

Chatter

Filters: All time • All activities • All types

Refresh • Expand All • View All

Upcoming & Overdue

Developer Edition

Testing App

Testing Phase

Testing data | Account

00001027 | Case

Account

Testing data

+ Follow

Edit

New Contact

New Case

Type

Phone

Website

Account Owner

Account Site

Industry

87654321

Lalji Prajapati

CSE

Related

Details

Assets

Account Owner

Lalji Prajapati

Rating

Account Name

Testing data

Phone

87654321

Parent Account

Fax

Account Number

Website

Account Site

Ticker Symbol

Type

Ownership

Industry

Employees

Service Request

* Case Subject

Account Data Testing

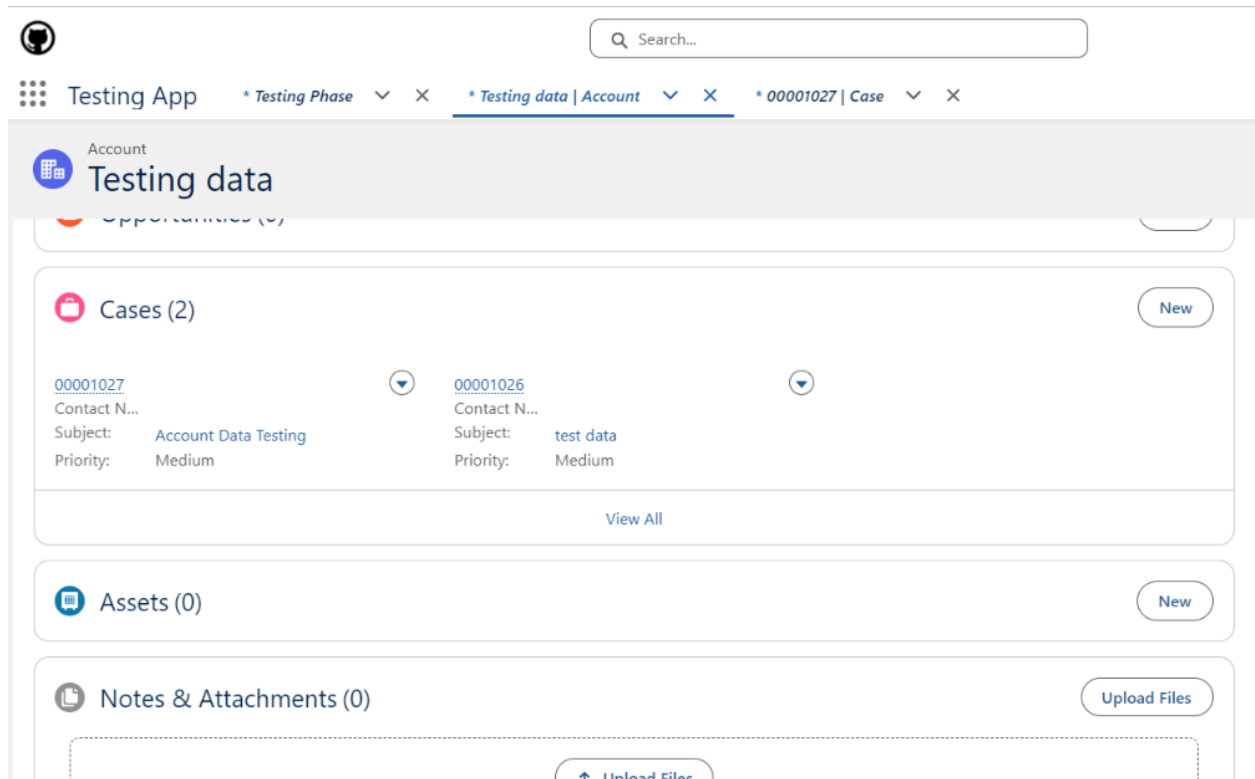
* Description

Account Data Testing

* Case Origin

Email

Next



Error Console:

Tired of “red box” error popups interrupting your testing? The new Error Console handles non-fatal errors in the background.

Seamless experience: You can keep working while errors are quietly logged for you to review later

Complete history: Even fatal errors that require a pop-up are logged here, giving you a full history to help with debugging

To enable it, go to Setup > User Interface and turn on “Use Error Console for error reporting in Lightning Experience” under Advanced Settings.

 **Error Console**

☐ Show Silent Errors

Clear List

All Errors

 Dec 2, 2025, 1:34:07 PM Customer Error

View Details

Uncaught Error in \$A.getCallback() [Customer Aura Error - Aura Callback]

 Dec 2, 2025, 1:34:02 PM Customer Error

View Details

Uncaught Action failed: c:customerGackGenerator\$controller\$throwError [Customer Aura Error - Default]

 Dec 2, 2025, 1:33:43 PM System Error

View Details

Uncaught AuraFriendlyError: ALERT Aura Error - Default

Lightning Out 2.0 improvements

Lightning Out 2.0 (introduced last release) gets several usability improvements. You can now configure host domains directly in the Lightning Out 2.0 App Manager. Add the external domain there, and to your Trusted Domains allowlist, in Session Settings.

Generated code blocks now include an app-id attribute for cleaner app identification.

In the Lightning Out 2.0 App Manager, you can now add Lightning web components (LWCs) with complex namespaces, such as those with mixed casing. Complex namespaces like `complexNs/lwcComponent` are now supported. Use either format: `complexNs/lwcComponent` (forward slash) or `complex_ns-lwc-component` (hyphen with underscores for namespace segments).

```

<template>
  <lightning-card title="Custom Card Component"
  icon-name="standard:lightning_component">
    <div class="slds-p-around_medium">
      <div class="slds-text-heading_small slds-m-bottom_medium">
        Lightning Web Component
      </div>

      <p class="slds-text-body_regular slds-m-bottom_medium">
        This component can receive messages from the host page and send
        responses back.
      </p>

      <div class="slds-m-bottom_medium">
        <lightning-button
          label="Send Message to Host Page"
          variant="brand"
          onclick="{handleSendMessage}"
          class="slds-m-bottom_small"
        >
      </lightning-button>

      <div class="slds-text-body_small slds-text-color_weak">
        Click this button to test communication from LWC to the host
        page.
      </div>
    </div>

    <div class="slds-text-heading_small slds-m-bottom_small">
      Received Messages ({messages.length})
    </div>

    <template if:true="{messages.length}">
      <div class="message-list">
        <template for:each="{messages}" for:item="message">
          <div key="{message.id}" class="message-item slds-box
slds-m-bottom_small">
            <div
class="slds-text-body_regular"><strong>Message:</strong>
{message.text}</div>

```



```

        <div class="slds-text-body_small slds-text-color_weak">
            <strong>Time:</strong> {message.timestamp}
        </div>
        <div class="slds-text-body_small slds-text-color_weak">
            <strong>Source:</strong> {message.source}
        </div>
    </div>
</template>
</div>
</template>

<template if:false="{messages.length}">
    <div class="slds-text-body_regular slds-text-color_weak">
        No messages received yet. Click "Send Message to LWC" on the
host page to start
        communication.
    </div>
</template>
</div>
</lightning-card>
</template>

//cardComponent.js
import { LightningElement, track } from "lwc";

export default class CardComponent extends LightningElement {
    @track messages = [];
    @track messageCounter = 0;

    connectedCallback() {
        // Listen for messages from the host page
        this.addEventListener("sendMessageToLWC", this.handleMessageFromHost);
    }

    disconnectedCallback() {
        // Clean up event listeners
        this.removeEventListener("sendMessageToLWC",
this.handleMessageFromHost);
    }
}

```

```

handleMessageFromHost = (event) => {
  console.log("Message received from host page:", event.detail);

  // Add message to our list
  this.messageCounter++;
  const newMessage = {
    id: this.messageCounter,
    text: event.detail.message,
    timestamp: event.detail.timestamp,
    source: event.detail.source,
  };

  this.messages = [...this.messages, newMessage];
};

handleSendMessage() {
  console.log("Sending message to host page...");

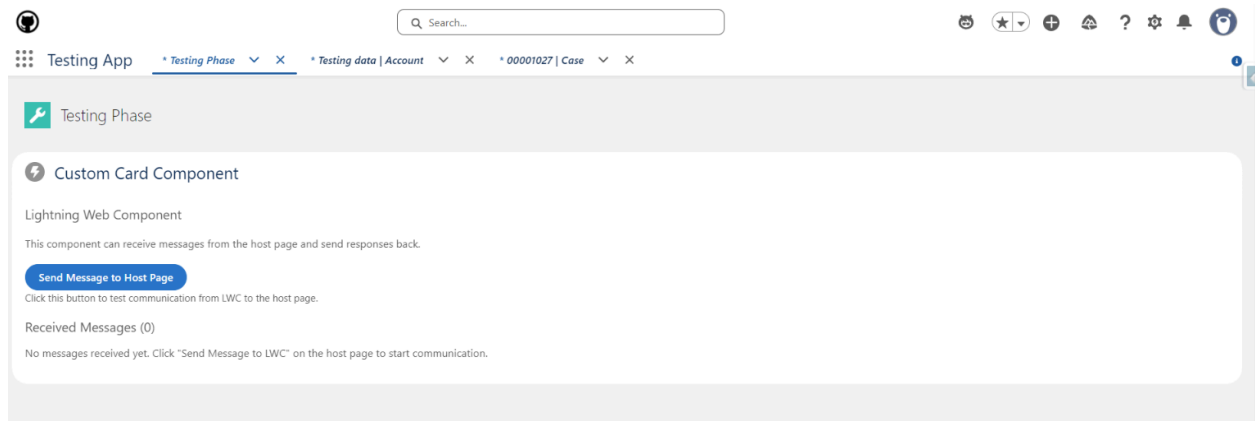
  const newMessage = `Message from LWC at ${new
Date().toLocaleTimeString()}`;

  const hostEvent = new CustomEvent("lwcMessageToHost", {
    detail: {
      message: newMessage,
      timestamp: new Date().toISOString(),
      component: "c-card-component",
    },
    bubbles: true,
    composed: true,
  });

  this.dispatchEvent(hostEvent);
  console.log("Message sent to host page");
}
}

```

Output:



Complex Template Expressions (Beta)

Spring '26 introduces Complex Template Expressions — a game-changing feature that lets you write JavaScript expressions directly in your LWC templates. Previously, even simple logic like concatenating strings or conditional display required that you create getter methods in your JavaScript file. Now, you can write expressions inline, dramatically reducing boilerplate code.

With template expressions, we can get rid of the getter and move the logic to the presentation layer instead:

```
{`Hello, ${firstName} ${lastName}!`}
```

The feature supports template literals, ternary operators, optional chaining, null coalescing, logical operators, arithmetic operations, array methods, and even inline event handlers. Below are some examples of using template expressions:

```
<template>
  <div class="slds-grid slds-wrap slds-gutters slds-p-around_medium">

    <div class="slds-col slds-size_1-of-1 slds-medium-size_1-of-2
slds-m-bottom_medium">
      <lightning-card title="Enter Employee Details"
icon-name="utility:user">
        <div class="slds-p-around_medium">
          <lightning-input
            label="First Name"
```

```
        name="firstName"
        value={formState.firstName}
        onChange={handleInputChange}>
</lightning-input>

<lightning-input
    label="Last Name"
    name="lastName"
    value={formState.lastName}
    onChange={handleInputChange}>
</lightning-input>

<lightning-input
    label="Department"
    name="department"
    value={formState.department}
    onChange={handleInputChange}>
</lightning-input>

<lightning-input
    type="number"
    label="Experience (Years)"
    name="experience"
    value={formState.experience}
    onChange={handleInputChange}>
</lightning-input>

<lightning-input
    type="toggle"
    label="Active Status"
    name="active"
    checked={formState.active}
    onChange={handleInputChange}
    message-toggle-active="Active"
    message-toggle-inactive="Inactive"
    class="slds-m-top_small">
</lightning-input>

<div class="slds-m-top_medium">
    <lightning-button
```

```

        variant="brand"
        label="Save Employee"
        onclick={handleSave}>
    </lightning-button>
</div>
</div>
</lightning-card>
</div>

<div class="slds-col slds-size_1-of-1 slds-medium-size_1-of-2">
    <template if:true={hasData}>
        <div class="employee-card state-animated">
            <div class="header">
                <span class="slds-flex
slds-alignment_vertical-center">
                    <lightning-icon icon-name="utility:identity"
size="small" class="slds-m-right_small icon-animated"></lightning-icon>
                    <span class="full-name">{fullName}</span>
                </span>
                <span class={statusClass}>{status}</span>
            </div>
            <hr class="slds-m-vertical_small" />
            <div class="details">
                <p class="detail-row">
                    <lightning-icon icon-name="utility:company"
size="x-small" class="slds-m-right_small icon-animated"></lightning-icon>
                    <span><strong>Department:</strong>
{employee.department}</span>
                </p>
                <p class="detail-row">
                    <lightning-icon
icon-name="utility:custom_apps" size="x-small" class="slds-m-right_small
icon-animated"></lightning-icon>
                    <span><strong>Experience:</strong>
{experienceLevel} ({employee.experience} Years)</span>
                </p>
                <p class="detail-row">
                    <lightning-icon icon-name="utility:moneybag"
size="x-small" class="slds-m-right_small icon-animated"></lightning-icon>

```

```

        <span><strong>Salary Band:</strong> <span
class={salaryClass}>{salaryBand}</span></span>
    </p>
</div>
</div>
</template>

<template if:false={hasData}>
    <div class="warning">
        No employee data saved yet. Fill out the form to
generate the dashboard.
    </div>
</template>
</div>
</template>

```

```

import { LightningElement, track } from 'lwc';
export default class EmployeeDashboard extends LightningElement {

    @track formState = {
        firstName: '',
        lastName: '',
        department: '',
        experience: '',
        active: false
    };

    @track employee = null;

    connectedCallback() {
        const savedData = localStorage.getItem('savedEmployeeData');
        if (savedData) {
            this.employee = JSON.parse(savedData);
        }
    }

    get hasData() {
        return this.employee !== null;
    }
}

```

```

    }

    handleInputChange(event) {
        const field = event.target.name;
        const value = (event.target.type === 'checkbox' ||
event.target.type === 'toggle')
            ? event.target.checked
            : event.target.value;

        this.formState[field] = value;
    }

    handleSave() {
        this.employee = { ...this.formState };
        localStorage.setItem('savedEmployeeData',
JSON.stringify(this.employee));

        this.formState = {
            firstName: '',
            lastName: '',
            department: '',
            experience: '',
            active: false
        };
    }

    get fullName() {
        return this.employee ? `${this.employee.firstName}
${this.employee.lastName}` : '';
    }

    get experienceLevel() {
        if (!this.employee || !this.employee.experience) return 'Junior';
        return Number(this.employee.experience) >= 5 ? 'Senior' :
'Junior';
    }

    get salaryBand() {
        if (!this.employee || !this.employee.experience) return 'Medium';
        return Number(this.employee.experience) >= 5 ? 'High' : 'Medium';
    }

```

```

    }

    get status() {
        if (!this.employee) return '';
        return this.employee.active ? 'Active' : 'Inactive';
    }

    get statusClass() {
        return this.employee?.active ? 'status-tag open' : 'status-tag
closed';
    }

    get salaryClass() {
        if (!this.employee) return '';
        return Number(this.employee.experience) >= 5 ? 'badge high' :
'badge medium';
    }
}

.employee-card {
    border-left: 5px solid #0176d3;
    border-radius: 6px;
    background: #ffffff;
    padding: 16px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}

.state-animated {
    animation: slideIn 0.4s ease-out forwards;
}

.icon-animated {
    display: inline-block;
    animation: iconPulse 2s infinite ease-in-out;
}

.header {
    display: flex;
    justify-content: space-between;

```



```
        align-items: center;
    }

    .full-name {
        font-size: 18px;
        font-weight: bold;
        color: #1b1b1b;
    }

    .detail-row {
        display: flex;
        align-items: center;
        margin: 10px 0;
        font-size: 14px;
    }

    .badge {
        padding: 2px 8px;
        border-radius: 12px;
        color: white;
        font-weight: bold;
        font-size: 11px;
        margin-left: 4px;
    }

    .high { background: #ba0517; }
    .medium { background: #fe9339; }
    .open { color: #0176d3; font-weight: bold; }
    .closed { color: #2e844a; font-weight: bold; }

    .warning {
        background: #fef1ee;
        color: #ba0517;
        padding: 12px;
        border-left: 4px solid #ba0517;
        border-radius: 4px;
        font-weight: bold;
    }
}
```

```

@keyframes slideIn {
  from {
    opacity: 0;
    transform: translateY(10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

@keyframes iconPulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.08); }
}

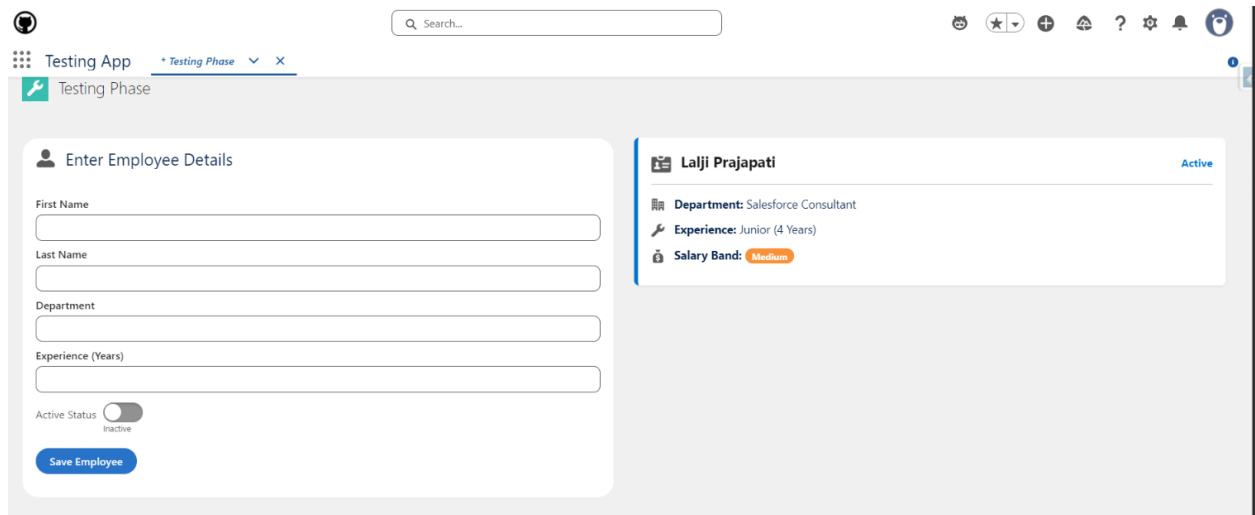
```

Output:

Before save data

The screenshot displays a web application interface for managing employee data. At the top, there is a search bar and a navigation menu with options like 'Testing App' and 'Testing Phase'. The main content area is divided into two sections. On the left, there is a form titled 'Enter Employee Details' with fields for 'First Name' (Lalji), 'Last Name' (Prajapati), 'Department' (Salesforce Consultant), 'Experience (Years)' (4), and 'Active Status' (Inactive). A 'Save Employee' button is at the bottom of this form. On the right, there is a summary card for 'Laksh Prajapati' showing 'Department: Sr. Salesforce Developer', 'Experience: Senior (5 Years)', and 'Salary Band: High'. The card also indicates the employee is 'Active'.

After save data



Lightning empty state and illustration components (Beta)

Spring '26 introduces two new base components — lightning-empty-state and lightning-illustration — that provide beautiful, SLDS-compliant empty state experiences with zero custom styling effort.

The lightning-empty-state component delivers a complete empty state pattern with an illustration, title, description slot, and call-to-action slot.

```
<template>
  <lightning-card title="Customer Cases" icon-name="standard:case">
    <template if:true={isLoading}>
      <lightning-spinner alternative-text="Loading"
size="medium"></lightning-spinner>
    </template>

    <template if:false={isLoading}>
      <template if:false={cases.length}>
        <div class="combined-empty-state-container">
```

```

        <img src={salesforceGif} alt="Branding Mascot"
class="branding-logo"/>
        <lightning-empty-state
            title="No Cases Found"
            class="slds-m-top_none compact-empty-state">

            <p slot="description" class="slds-text-color_weak
slds-m-top_none">

                This customer doesn't have any support cases
                yet.

                Click <strong>Create Case</strong> to register
                the first service request.
            </p>

            <div slot="cta">
                <lightning-button
                    label="Create Case"
                    variant="brand"
                    icon-name="utility:add"
                    onclick={handleCreateCase}>
                </lightning-button>
            </div>
        </lightning-empty-state>

    </div>
</template>
<template if:true={cases.length}>
    <div class="slds-p-around_small">
        <template for:each={cases} for:item="caseRecord">
            <div key={caseRecord.Id} class="case-row">
                <strong>{caseRecord.CaseNumber}</strong> -
                {caseRecord.Subject}
            </div>
        </template>
    </div>
</template>
</div>
</template>

</template>
</lightning-card>
</template>

```

```

import { LightningElement, api, wire } from 'lwc';
import getCases from '@salesforce/apex/CaseService.getCases';
import SALESFORCE_GIF from '@salesforce/resourceUrl/SalesforceAnimation';
export default class CustomerCases extends LightningElement {

    salesforceGif = SALESFORCE_GIF;
    @api recordId;

    cases = [];

    error;

    isLoading = true;

    @wire(getCases, {
        accountId: '$recordId'
    })
    wiredCases({ data, error }) {

        this.isLoading = false;

        if (data) {

            this.cases = data;
            this.error = undefined;

        } else if (error) {

            this.error = error;
            this.cases = [];

        }

    }

    handleCreateCase() {

```

```
        // Navigate to Flow or New Record Page depending on enterprise  
requirement
```

```
    }
```

```
}
```

```
.combined-empty-state-container {  
    display: flex;  
    flex-direction: column;  
    justify-content: center;  
    align-items: center;  
    padding: 1.5rem 1rem;  
    text-align: center;  
    width: 100%;  
}
```

```
.branding-logo {  
    width: 140px;  
    height: auto;  
    margin-bottom: 0.25rem;  
    display: block;  
}
```

```
.combined-empty-state-container lightning-empty-state {  
    width: 100%;  
    max-width: 440px;  
    display: block;  
}
```

```
.compact-empty-state {  
    --slds-c-empty-state-image-margin-bottom: 0rem;  
}
```

```
public with sharing class CaseService {  
  
    @AuraEnabled(cacheable=true)  
    public static List<Case> getCases(Id accountId) {  
  
        if (accountId == null) {  
            return new List<Case>();  
        }  
    }  
}
```

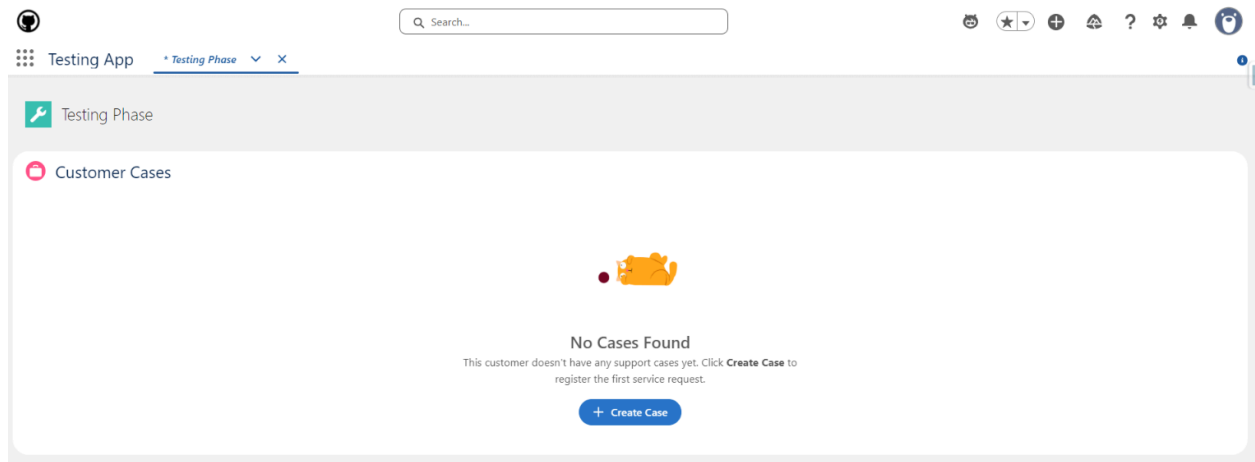
```

    }

    return [
        SELECT
            Id,
            CaseNumber,
            Subject,
            Status,
            Priority,
            CreatedDate
        FROM Case
        WHERE AccountId = :accountId
        ORDER BY CreatedDate DESC
    ];
}
}

```

Output:



Apex updates in the Spring '26 release

Apex is also getting several developer improvements in the Spring '26 release.

Apex cursors:

Apex cursors (now GA in API v66.0) let you work with large SOQL result sets in manageable chunks — without the rigidity of batch Apex. Create a cursor once, then fetch records from any position, forwards or backwards.

Below is an example that shows basic usage:

```
public with sharing class AccountBatchProcessorQueueable implements
Queueable {

    private Database.Cursor accountCursor;
    private Integer currentPosition;
    private static final Integer CHUNK_SIZE = 1000;

    /**
     * Initial constructor called to start the pipeline execution context.
     */
    public AccountBatchProcessorQueueable() {

        // Generate the cursor scope via a valid SOQL query string
        this.accountCursor = Database.getCursor('SELECT Id, Name,
Description, AnnualRevenue FROM Account WHERE Industry = \'Technology\');
        this.currentPosition = 0;
    }

    /**
     * Overloaded constructor used during asynchronous chaining to
    maintain progress tracking state.
     */
    public AccountBatchProcessorQueueable(Database.Cursor existingCursor,
Integer nextPosition) {
        this.accountCursor = existingCursor;
        this.currentPosition = nextPosition;
    }

    /**
```



```

    * Main execution method invoked asynchronously by the core
transaction engine.
    */

    public void execute(QueueableContext context) {

        Integer totalRows = this.accountCursor.getNumRecords();

        // Safety execution fallback guard
        if (this.currentPosition >= totalRows) {
            return;
        }

        // Fetch a specific chunk slice forward based on currentPosition
marker
        List<Account> accountsChunk =
this.accountCursor.fetch(this.currentPosition, CHUNK_SIZE);

        List<Account> accountsToUpdate = new List<Account>();

        for (Account acc : accountsChunk) {
            acc.Description = 'Verified Profile Review via Spring 26 Apex
Cursors.';
            accountsToUpdate.add(acc);
        }

        if (!accountsToUpdate.isEmpty()) {
            update Security.stripInaccessible(AccessType.UPDATABLE,
accountsToUpdate).getRecords();
        }

        // Update pointer position indicator tracks
this.currentPosition += accountsChunk.size();

        // FIX: Changed getNumRows() to getNumRecords() here as well
        if (this.currentPosition < this.accountCursor.getNumRecords() &&
!Test.isRunningTest()) {
            System.enqueueJob(new
AccountBatchProcessorQueueable(this.accountCursor, this.currentPosition));
        }
    }

```

```
}  
}
```

Processing large datasets with Queueable:

Cursors can be serialized and passed between Queueable jobs, perfect for chaining through millions of records.

Here is an example code showing this in action:

```
public class ProcessRecordsJob implements Queueable {  
  
    private Database.Cursor cursor;  
    private Integer position;  
    private static final Integer CHUNK_SIZE = 20;  
  
    public ProcessRecordsJob(String query) {  
        cursor = Database.getCursor(query);  
        position = 0;  
    }  
  
    public void execute(QueueableContext ctx) {  
  
        Integer total = cursor.getNumRecords();  
  
        Integer remaining = total - position;  
  
        if (remaining <= 0) {  
            return;  
        }  
  
        Integer count = Math.min(CHUNK_SIZE, remaining);  
  
        List<SObject> scope = cursor.fetch(position, count);  
  
        position += scope.size();  
  
        System.debug('Processing ' + scope.size() +
```

```

        ' records. Position=' + position);

    if (position < total) {
        System.enqueueJob(this);
    }
}
}

```

Pagination for UI:

Cursors support `@AuraEnabled` serialization — pass them between LWC and Apex for seamless page navigation.

Here is an example showing pagination in action:

```

public with sharing class PaginationResult {
    public class PaginationResultMethod {
        @AuraEnabled public Database.Cursor cursor { get; set; }
        @AuraEnabled public List<SObject> records { get; set; }
        @AuraEnabled public Integer currentPage { get; set; }
        @AuraEnabled public Integer totalPages { get; set; }
    }

    @AuraEnabled
    public static PaginationResultMethod getPage(Database.Cursor cursor,
Integer page, Integer size) {
        Integer offset = (page - 1) * size;
        PaginationResultMethod result = new PaginationResultMethod();
        if (cursor == null) {
            cursor = Database.getCursor('SELECT Id, CaseNumber, Subject,
Status, Priority FROM Case ORDER BY CreatedDate DESC');
        }
        result.cursor = cursor;
        result.records = cursor.fetch(offset, size);
        result.currentPage = page;
        result.totalPages = (Integer) Math.ceil((Decimal)
cursor.getNumRecords() / size);
        return result;
    }
}

```

Extract picklist values based on record type:

Spring '26 introduces a powerful new Apex method that eliminates a long-standing pain point for Salesforce Developers: `ConnectApi.RecordUi.getPicklistValuesByRecordType(objectApiName, recordTypeId)`.

Previously, retrieving record type-specific picklist values in Apex required cumbersome callouts to the UI API — but no more. With this single method call, you can now fetch all picklist values for a specific record type directly in Apex, making it incredibly efficient for building dynamic forms, especially those with dependent picklist hierarchies like Continent > case > Department__c.

Here is an example showing the new method in action. The returned `PicklistValuesCollection` contains all picklist fields for that record type in one request, including their dependent relationships and default values — no multiple API calls, no callouts, just clean native Apex.

```
public with sharing class EmployeeController {

    @AuraEnabled(cacheable=true)
    public static List<String> getDepartmentValues(Id recordTypeId) {

        List<String> departments = new List<String>();

        ConnectApi.PicklistValuesCollection collection =
        ConnectApi.RecordUi.getPicklistValuesByRecordType('Employee__c',
        recordTypeId);

        ConnectApi.PicklistValues fieldValues =
        collection.picklistFieldValues.get('Department__c');

        if(fieldValues != null){
            for(ConnectApi.PicklistValue value : fieldValues.values){
                departments.add(value.label);
            }
        }

        return departments;
    }
}
```

Visualforce PDF rendering service with Apex Blob.toPdf():

Starting with Spring '26 (enforced in Summer '26), the Blob.toPdf() method now leverages the same powerful rendering engine as Visualforce PDFs, bringing consistent rendering, expanded font support, and full multibyte character support for CJK languages (Chinese, Japanese, Korean), Thai, Arabic, and more. The API remains unchanged — Blob.toPdf() still accepts any HTML string and returns a blob — but the underlying engine delivers a modern, sans-serif default font and reliable international character rendering. This makes it perfect for generating professional documents like certificates, invoices, and reports that serve global audiences.

Note: If your existing PDFs rely on the old serif default font, you may notice changes in line breaks and character spacing. To maintain backward compatibility, explicitly set font-family: serif; in your CSS. To prepare, review this update in Setup > Release Updates and test in your sandbox before the Summer '26 enforcement date (check Trust Status for your instance's specific upgrade timeline).

```
/**
 * @description Advanced global invoice rendering framework engine
utilizing the upgraded
 * Spring '26 Blob.toPdf() engine for native multibyte character support
and high-fidelity layouts.
 */
public with sharing class GlobalInvoiceService {

    /**
     * @description Generates an itemized enterprise invoice PDF based on
an account record dataset.
     * @param accountId Target Account record ID
     * @return Attachment The created PDF file attachment record reference
     */
    public static Attachment generateGlobalInvoice(Id accountId) {
        // Fetch extended customer info to expand billing coverage details
        Account acc = [
            SELECT Name, BillingStreet, BillingCity, BillingState,
BillingPostalCode, BillingCountry,
```

```

        AccountNumber, Phone, Website
    FROM Account
    WHERE Id = :accountId
    LIMIT 1
];

// Localized and parameterized metadata strings
String invoiceTitle = 'कर इनवाइस / TAX INVOICE';
String invoiceNumber = 'INV-' +
String.valueOf(Crypto.getRandomInteger()).replace('-', '');
String fileDateSuffix = DateTime.now().format('yyyy-MM-dd');
String issueDate = Date.today().format();
String dueDate = Date.today().addDays(30).format();

// 1. Constructing a highly styled, modern CSS invoice canvas
framework
String htmlContent = '<html>' +
    '<head>' +
    '    <style>' +
    '        @page { size: a4; margin: 0mm; }' +
    '        body { font-family: "Arial", sans-serif; margin: 0; padding: 40px; color: #2b2b2b; line-height: 1.4; background-color: #ffffff; }' +
    '        ' +
    '        /* Branding Layout Utilities */' +
    '        .invoice-header { display: block; margin-bottom: 40px; border-bottom: 3px solid #0176d3; padding-bottom: 20px; }' +
    '        .company-section { float: left; width: 50%; }' +
    '        .meta-section { float: right; width: 50%; text-align: right; }' +
    '        .clear { clear: both; }' +
    '        ' +
    '        .brand-title { font-size: 28px; font-weight: bold; color: #0176d3; text-transform: uppercase; margin: 0 0 5px 0; }' +
    '        .company-name { font-size: 18px; font-weight: bold; color: #1b1b1b; margin: 0 0 5px 0; }' +
    '        .text-muted { color: #696969; font-size: 12px; }' +
    '        ' +
    '        /* Bill To / Info Grid */' +
    '        .details-grid { margin-bottom: 40px; }' +

```

```

        .bill-to { float: left; width: 60%; }' +
        .bill-details { float: right; width: 40%; text-align:
right; font-size: 13px; }' +
        ' +
        /* Modern Data Table Styling */' +
        table { width: 100%; border-collapse: collapse;
margin-bottom: 30px; font-size: 13px; }' +
        th { background-color: #f3f3f3; color: #1b1b1b;
font-weight: bold; padding: 12px 10px; text-align: left; border-bottom:
2px solid #e5e5e5; }' +
        td { padding: 12px 10px; border-bottom: 1px solid
#eeeeee; color: #3e3e3e; }' +
        .text-right { text-align: right; }' +
        ' +
        /* Financial Summaries */' +
        .summary-wrapper { float: right; width: 35%;
margin-top: 10px; }' +
        .summary-row { padding: 6px 0; font-size: 13px;
border-bottom: 1px solid #f3f3f3; }' +
        .total-row { font-size: 16px; font-weight: bold;
color: #0176d3; border-top: 2px solid #0176d3; border-bottom: none;
padding-top: 10px; margin-top: 5px; }' +
        ' +
        .footer { position: fixed; bottom: 30px; left: 40px;
right: 40px; text-align: center; font-size: 10px; color: #747474;
border-top: 1px solid #e5e5e5; padding-top: 15px; }' +
    </style>' +
</head>' +
<body>' +
    ' +
    ' +
    <div class="invoice-header">' +
        <div class="company-section">' +
            <div class="company-name">CLOUD SYSTEM SOLUTIONS
LTD</div>' +
            <div class="text-muted">Hazratganj Main Road,
Lucknow, UP, 226001</div>' +
            <div class="text-muted">Contact:
support@cloudsystem.com | www.cloudsystem.com</div>' +
            </div>' +

```

```

        <div class="meta-section">' +
        <div class="brand-title">' + invoiceTitle +
'</div>' +
        <div style="font-size: 14px; margin-top:
5px;"><strong>Invoice #:</strong> ' + invoiceNumber + '</div>' +
        </div>' +
        <div class="clear"></div>' +
        </div>' +
        ' +
        ' +
        <div class="details-grid">' +
        <div class="bill-to">' +
        <div style="font-weight: bold; font-size: 12px;
color: #0176d3; margin-bottom: 5px; text-transform: uppercase;">Bill
To:</div>' +
        <div style="font-size: 15px; font-weight: bold;
margin-bottom: 3px;">' + acc.Name + '</div>' +
        <div class="text-muted">' + (acc.BillingStreet !=
null ? acc.BillingStreet + '<br/>' : '') + '</div>' +
        <div class="text-muted">' + (acc.BillingCity !=
null ? acc.BillingCity + ', ' : '') + (acc.BillingState != null ?
acc.BillingState + ' ' : '') + (acc.BillingPostalCode != null ?
acc.BillingPostalCode : '') + '</div>' +
        <div class="text-muted">' + (acc.BillingCountry
!= null ? acc.BillingCountry : '') + '</div>' +
        <div class="text-muted" style="margin-top:
5px;"><strong>Client Account Reference:</strong> ' + (acc.AccountNumber !=
null ? acc.AccountNumber : 'N/A') + '</div>' +
        </div>' +
        <div class="bill-details">' +
        <div><strong>Issue Date:</strong> ' + issueDate +
'</div>' +
        <div style="margin-top: 5px;"><strong>Due
Date:</strong> ' + dueDate + '</div>' +
        <div style="margin-top: 5px;"><strong>Payment
Terms:</strong> Net 30 Days</div>' +
        </div>' +
        <div class="clear"></div>' +
        </div>' +
        ' +

```



```

        '    ' +
        '    <table>' +
        '        <thead>' +
        '            <tr>' +
        '                <th style="width: 50%;">Description /
विवरण</th>' +
        '                <th style="width: 15%;"
class="text-right">Qty</th>' +
        '                <th style="width: 15%;"
class="text-right">Unit Price</th>' +
        '                <th style="width: 20%;"
class="text-right">Amount (INR)</th>' +
        '            </tr>' +
        '        </thead>' +
        '        <tbody>' +
        '            <tr>' +
        '                <td>Enterprise Architecture Deployment
Consulting (Q2 Production Phase)</td>' +
        '                <td class="text-right">1</td>' +
        '                <td class="text-right">₹1,50,000.00</td>' +
        '                <td class="text-right">₹1,50,000.00</td>' +
        '            </tr>' +
        '            <tr>' +
        '                <td>Managed Platform User Licensing Services
Suite (Premium Tier Allocation)</td>' +
        '                <td class="text-right">25</td>' +
        '                <td class="text-right">₹2,000.00</td>' +
        '                <td class="text-right">₹50,000.00</td>' +
        '            </tr>' +
        '        </tbody>' +
        '    </table>' +
        '    ' +
        '    ' +
        '    <div class="summary-wrapper">' +
        '        <div class="summary-row">' +
        '            <span style="float: left;">Subtotal:</span>' +
        '            <span style="float: right;">₹2,000,000.00</span>'
+
        '        <div class="clear"></div>' +
        '    </div>' +

```

```

        '          <div class="summary-row">' +
        '          <span style="float: left;">Tax Integration
(18%):</span>' +
        '          <span style="float: right;">₹36,000.00</span>' +
        '          <div class="clear"></div>' +
        '        </div>' +
        '        <div class="summary-row total-row">' +
        '          <span style="float: left;">Total Due:</span>' +
        '          <span style="float: right;">₹2,36,000.00</span>'
+
        '          <div class="clear"></div>' +
        '        </div>' +
        '      </div>' +
        '      <div class="clear"></div>' +
        '      ' +
        '      ' +
        '      <div class="footer">' +
        '        Thank you for your business. Please remit payments
directly to our corporate bank node routing profiles.<br/>' +
        '        <span style="font-size: 8px; color: #a1a1a1;
margin-top: 5px; display: block;">Generated natively under active
compliance via Spring 26 Core Blob.toPdf() Rendering Pipeline.</span>' +
        '      </div>' +
        '</body>' +
        '</html>';

// Convert the dynamic canvas layout directly to binary PDF format
Blob pdfBlob = Blob.toPdf(htmlContent);

Attachment pdfAttachment = new Attachment();
pdfAttachment.ParentId = acc.Id;
pdfAttachment.Name = 'Invoice_' + invoiceNumber + '_' +
fileDateSuffix + '.pdf';
pdfAttachment.Body = pdfBlob;
pdfAttachment.ContentType = 'application/pdf';

insert Security.stripInaccessible(AccessType.CREATABLE, new
List<Attachment>{pdfAttachment}).getRecords();
return pdfAttachment;
}

```

}

Output:

The screenshot displays a Salesforce interface. The top navigation bar includes a search bar and various utility icons. The main content area shows a 'Testing data' account with sections for Assets (0), Notes & Attachments (1), Partners (0), and Invoices (0). The 'Notes & Attachments' section contains a PDF attachment named 'Invoice_INV-1050325879_2026-06-28.pdf' dated 28-Jun-2026.

Below the Salesforce interface, a browser window shows a 'servlet.FileDownload' page. The page displays a 'TAX INVOICE' for 'CLOUD SYSTEM SOLUTIONS LTD'. The invoice details include:

- Company:** CLOUD SYSTEM SOLUTIONS LTD, Hazratganj Main Road, Lucknow, UP, 226001, Contact: support@cloudsystem.com | www.cloudsystem.com
- Invoice #:** INV-884326293
- Issue Date:** 28/06/2026
- Due Date:** 28/07/2026
- Payment Terms:** Net 30 Days
- Client Account Reference:** N/A

Description /	Qty	Unit Price	Amount (INR)
Enterprise Architecture Deployment Consulting (Q2 Production Phase)	1	1,50,000.00	1,50,000.00
Managed Platform User Licensing Services Suite (Premium Tier Allocation)	25	2,000.00	50,000.00
Subtotal:			2,00,000.00
Tax Integration (18%):			36,000.00
Total Due:			2,36,000.00

Miscellaneous Apex updates:

In addition to these important updates, there are a few interesting changes to Apex:

```
public with sharing class OrderAgentActions {  
  
    @InvocableMethod(label='Get Customer Orders')  
    public static List<String> getOrders(List<Id> accountIds) {
```

```
List<String> results = new List<String>();

for(Account acc : [SELECT Name FROM Account WHERE Id IN
:accountIds]){

    results.add('Orders for ' + acc.Name);
}

return results;
}
```